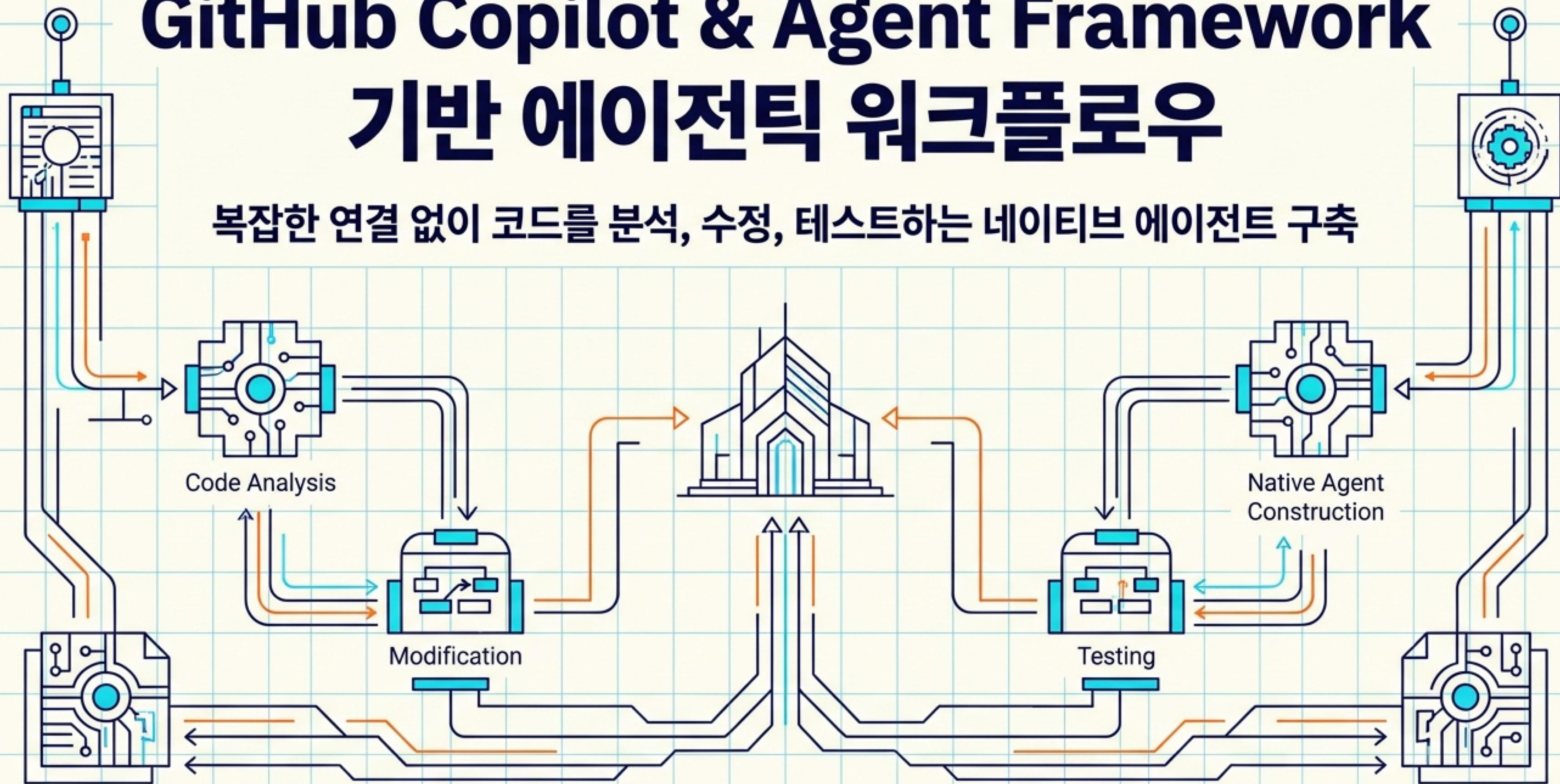


GitHub Copilot & Agent Framework

기반 에이전틱 워크플로우

복잡한 연결 없이 코드를 분석, 수정, 테스트하는 네이티브 에이전트 구축



후원사

다이아몬드



플래티넘



골드 

go deploy

STK



실버 



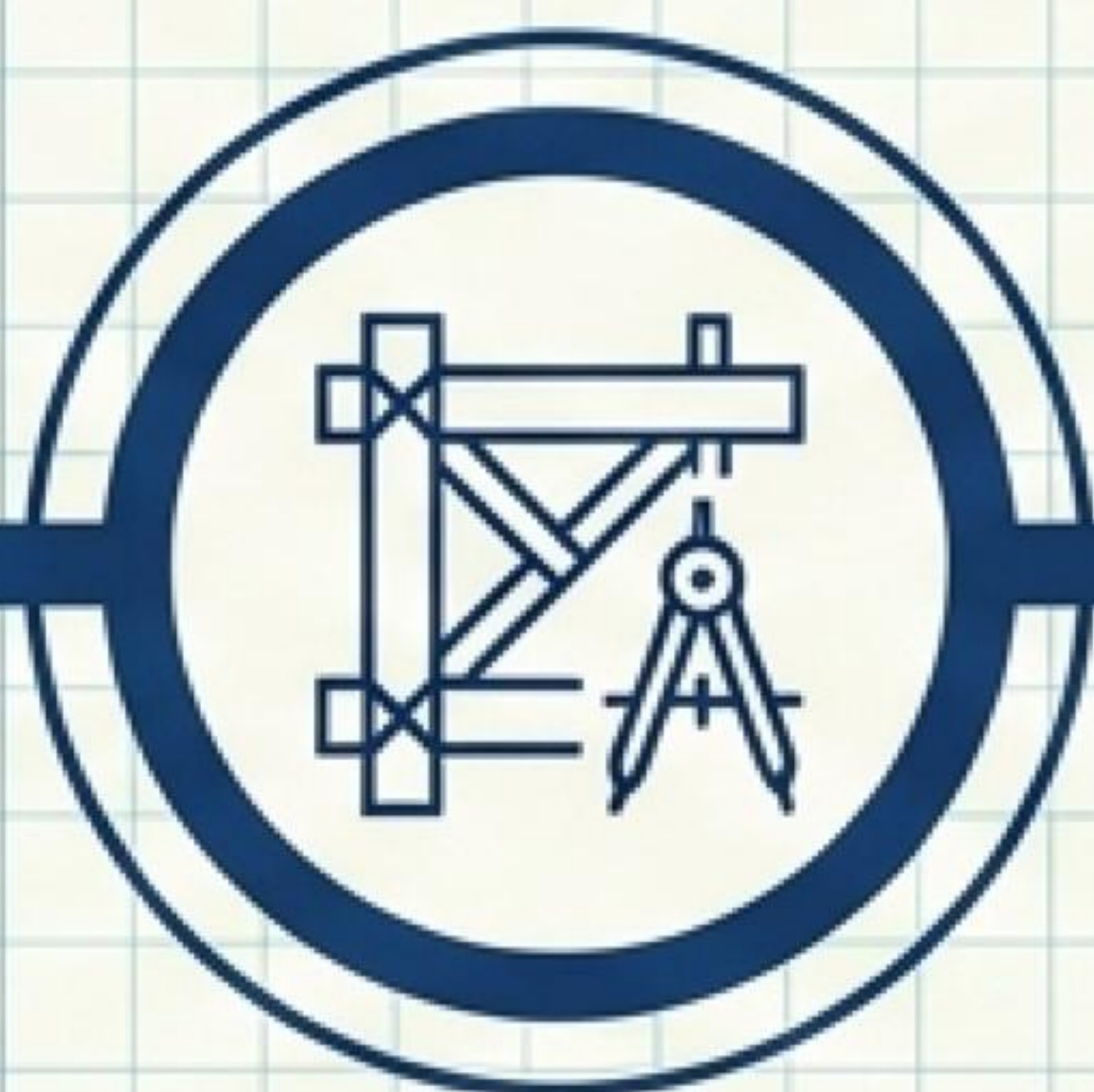
브론즈 



오늘의 여정



오픈 &
문제 제기



Framework
아키텍처



Workflow
오케스트레이션

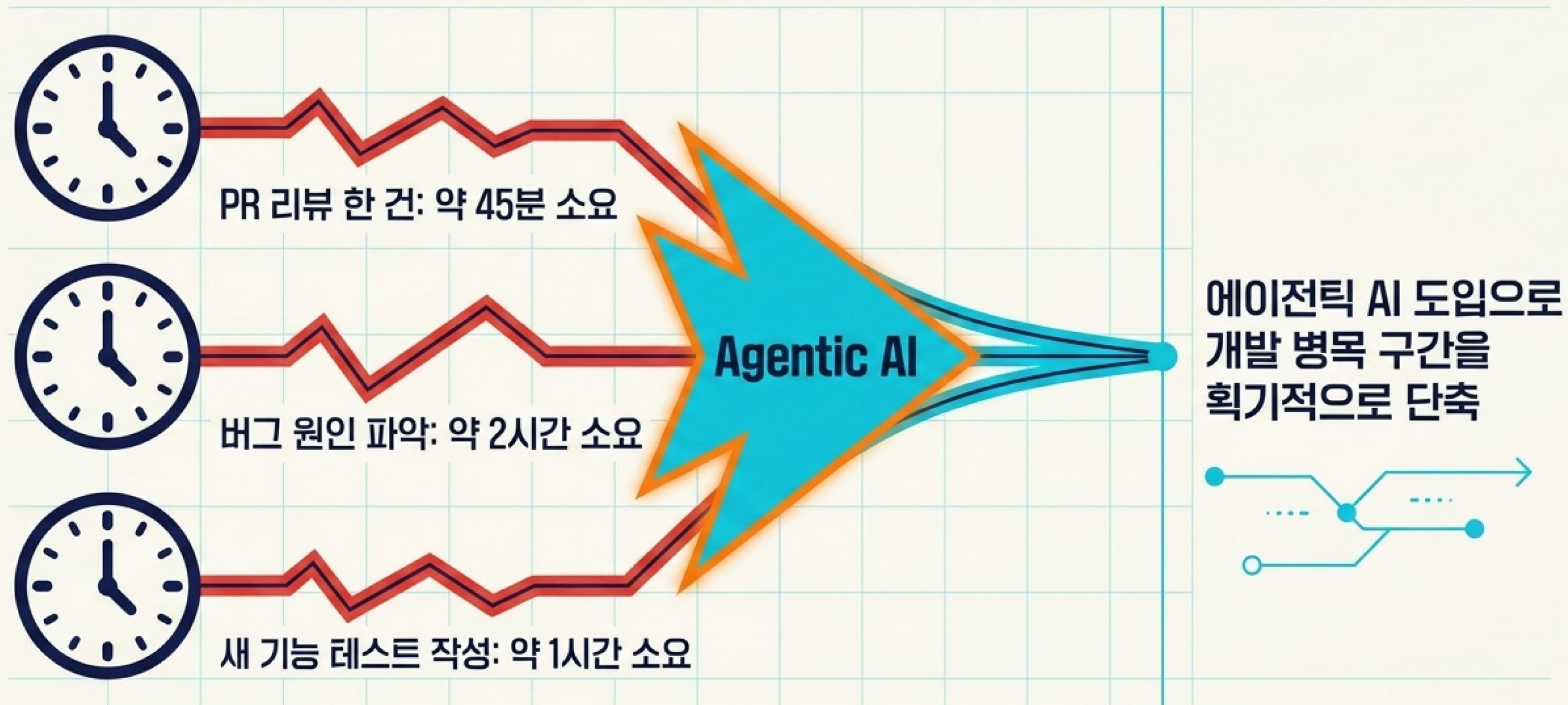


Copilot SDK
통합



엔터프라이즈
에코시스템

개발 생산성의 새로운 패러다임



반복적이고 소모적인 인지 작업을 자율 에이전트에게 위임

Agent vs Workflow: 무엇을 선택할 것인가



Agent (에이전트)

- 태스크가 개방형이거나 대화형일 때 적합
- 자율적인 도구(Tool) 사용과 계획 수립 필요
- 단일 LLM 호출로 문제 해결이 가능한 단일 도메인

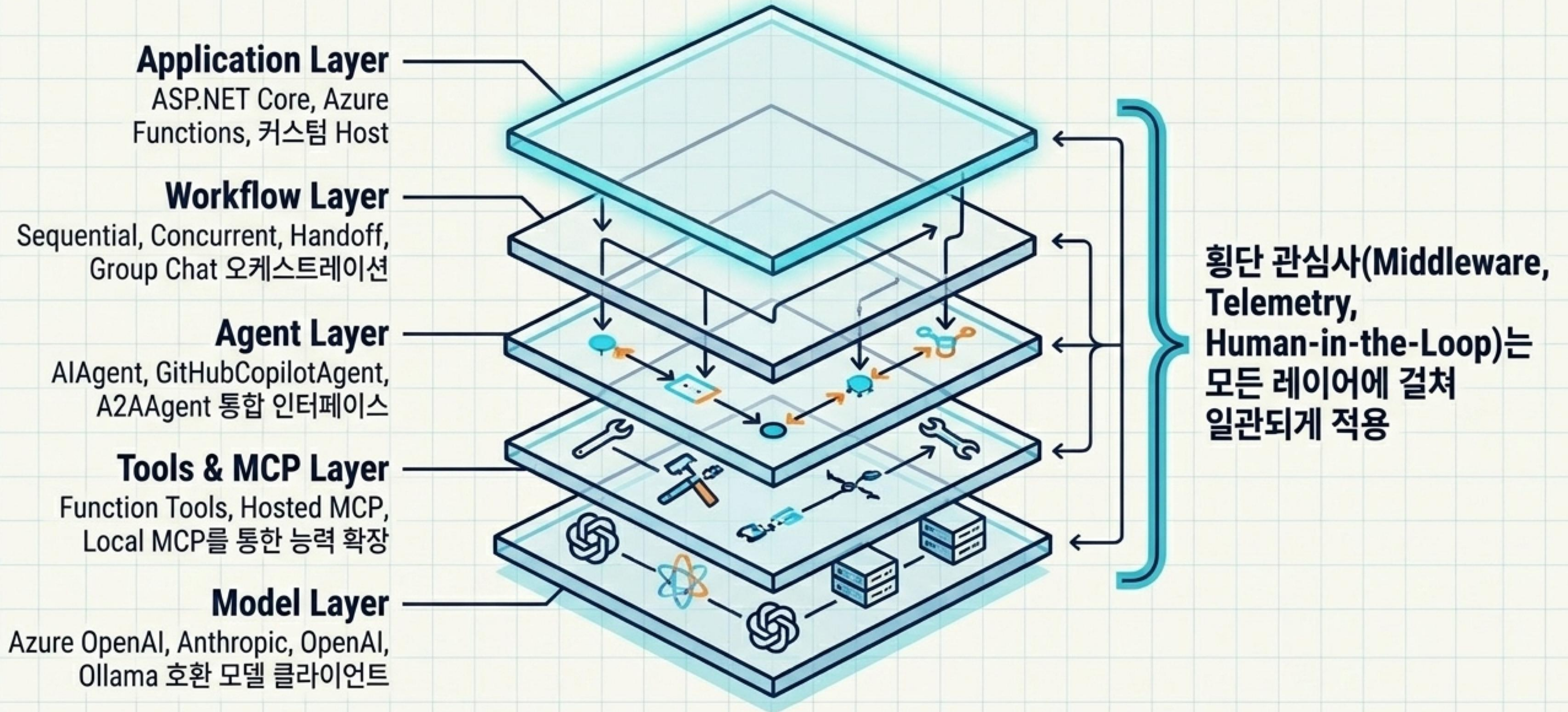


Workflow (워크플로우)

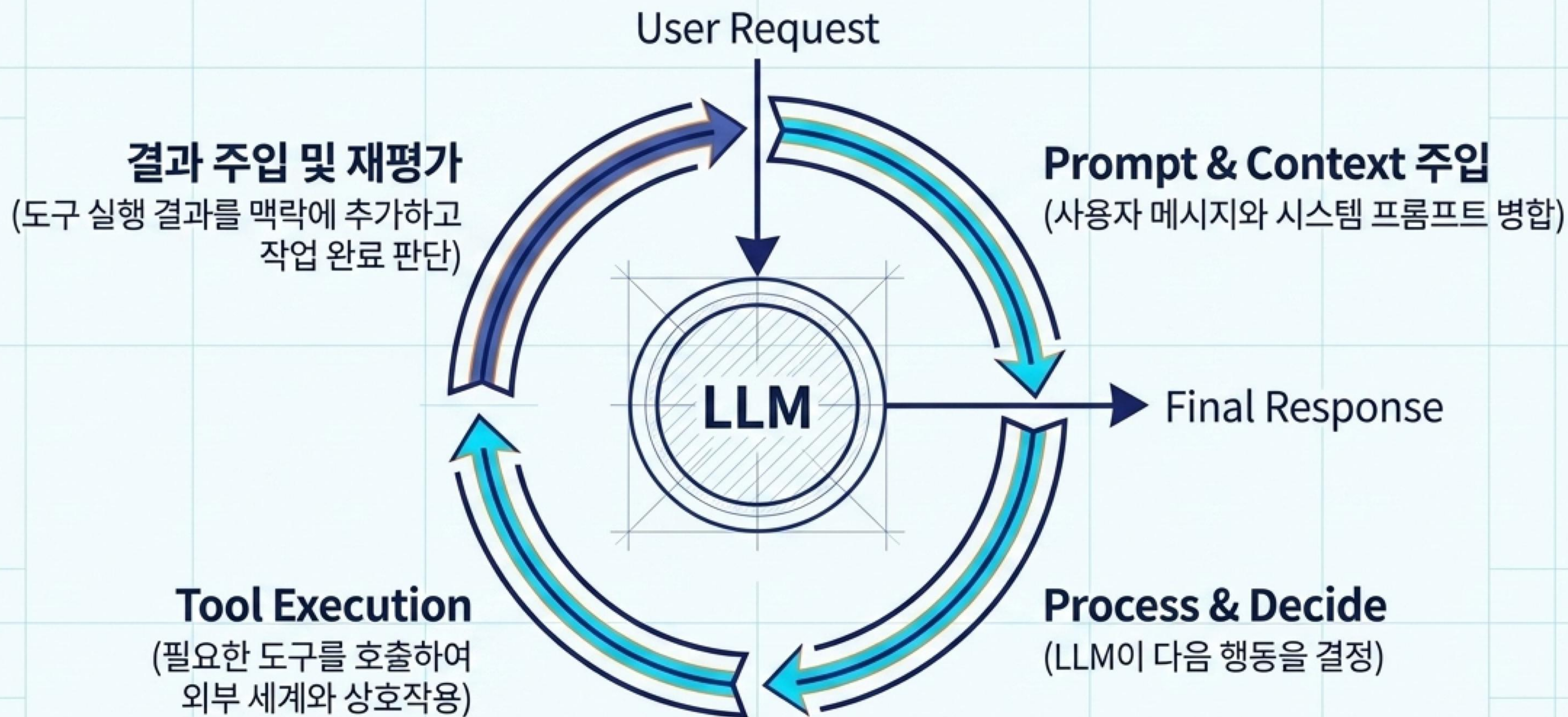
- 프로세스의 실행 단계가 명확히 정의된 경우
- 실행 순서와 데이터 흐름에 대한 명시적 제어 필요
- 여러 에이전트와 함수의 조율이 필수적인 복잡한 비즈니스 로직

함수로 해결할 수 있는 작업은 함수로, 지능적 조율이 필요할 때 에이전트와 워크플로우를 결합

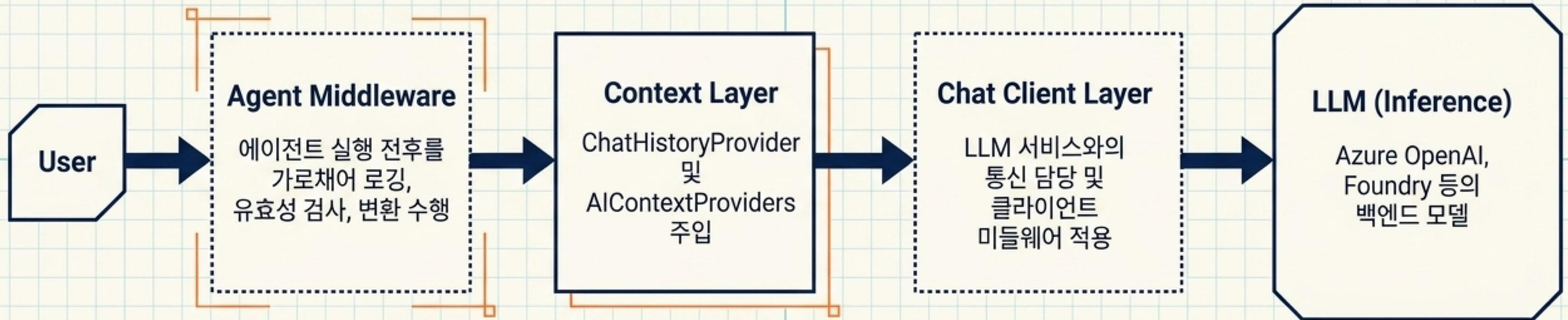
Agent Framework 전체 아키텍처



에이전틱 루프의 핵심 구조

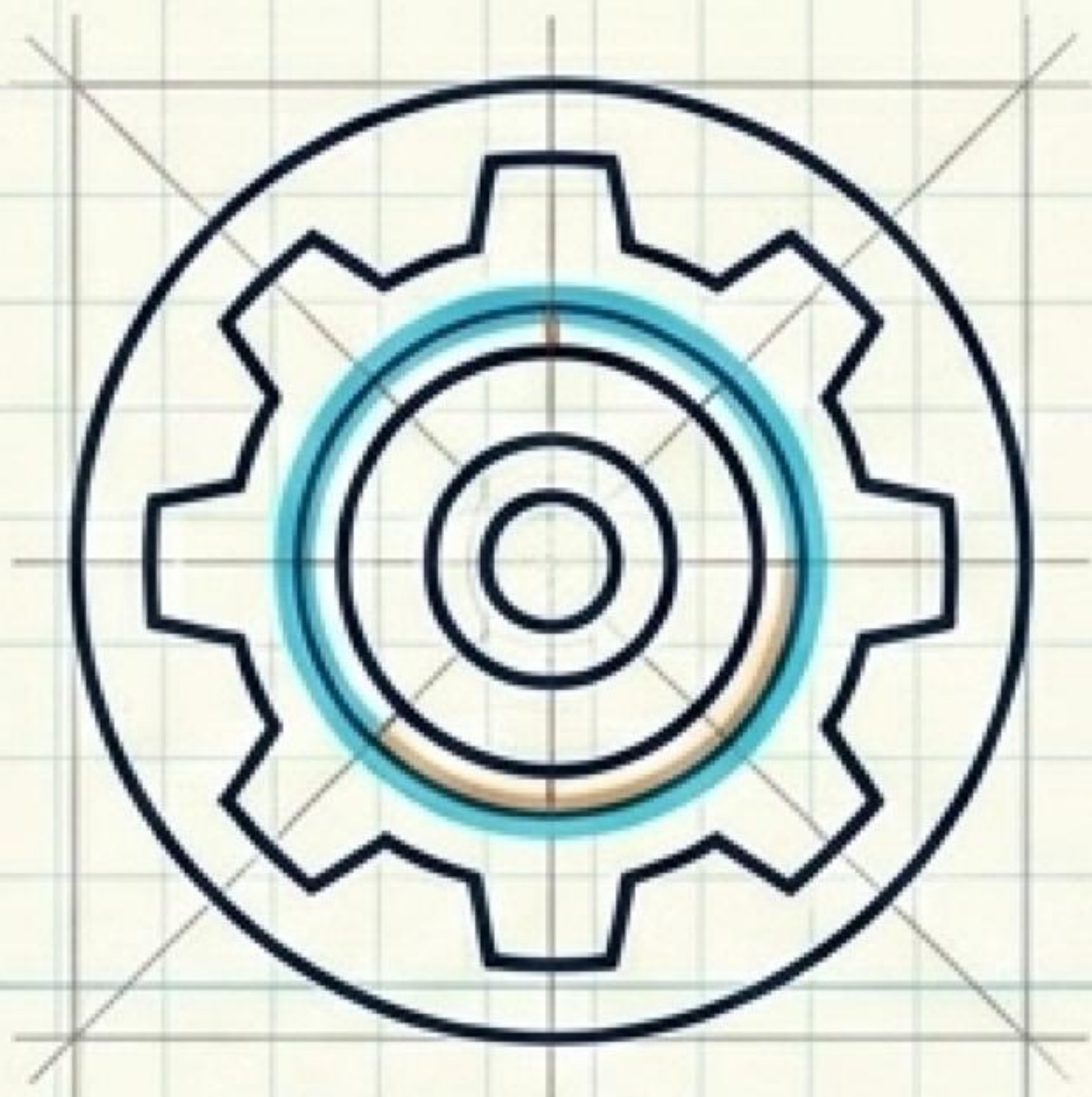


파이프라인 아키텍처 심층 분석



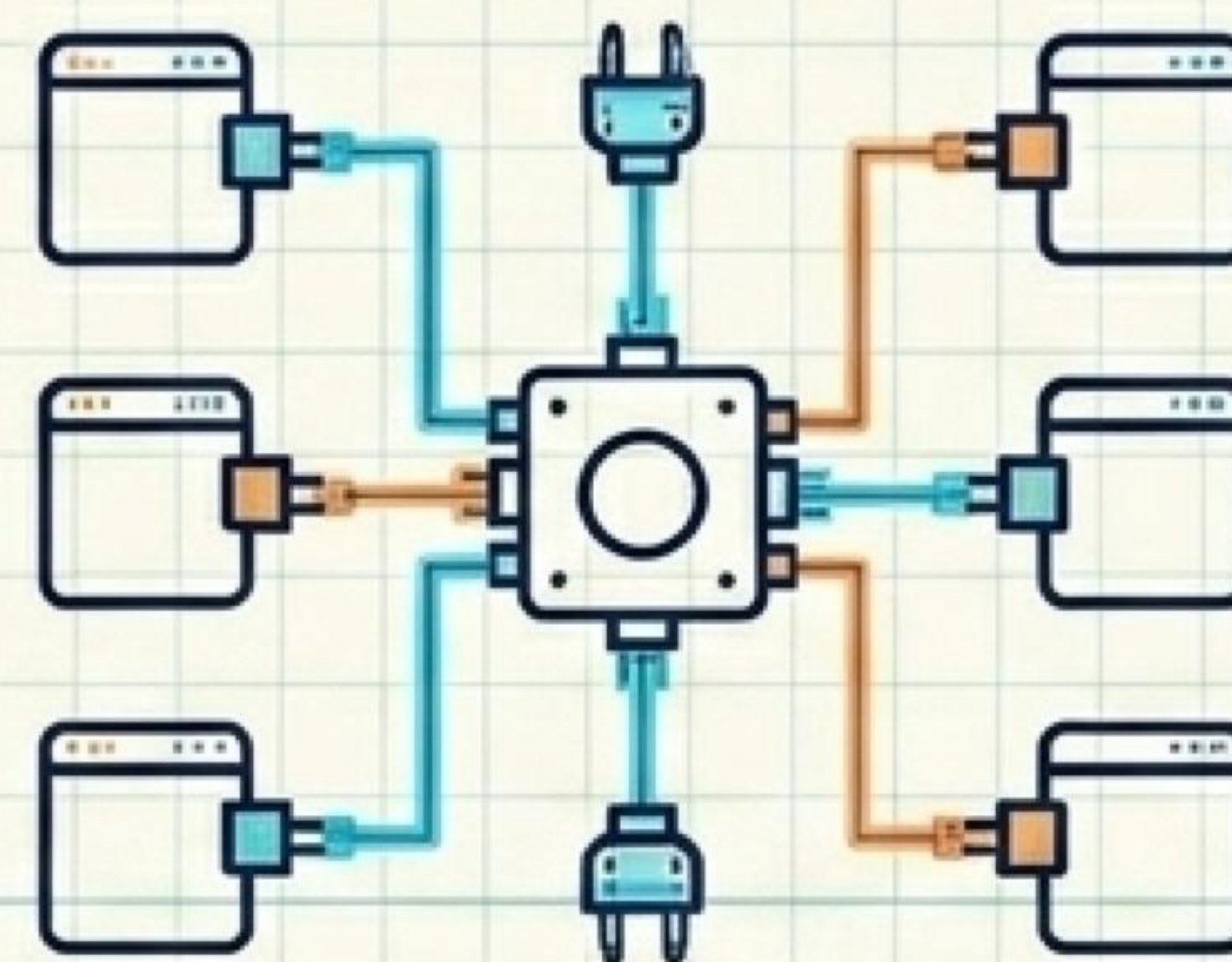
각 레이어는 독립적으로 확장 가능하며 비즈니스 로직을 변경하지 않고 동작 제어

에이전트의 능력 확장: Tools & MCP



Function Tools

- 메서드에 [Description] 속성을 부여하여 에이전트가 자동 인식
- 날씨 조회, 데이터베이스 쿼리 등 직접 구현된 내부 로직 호출

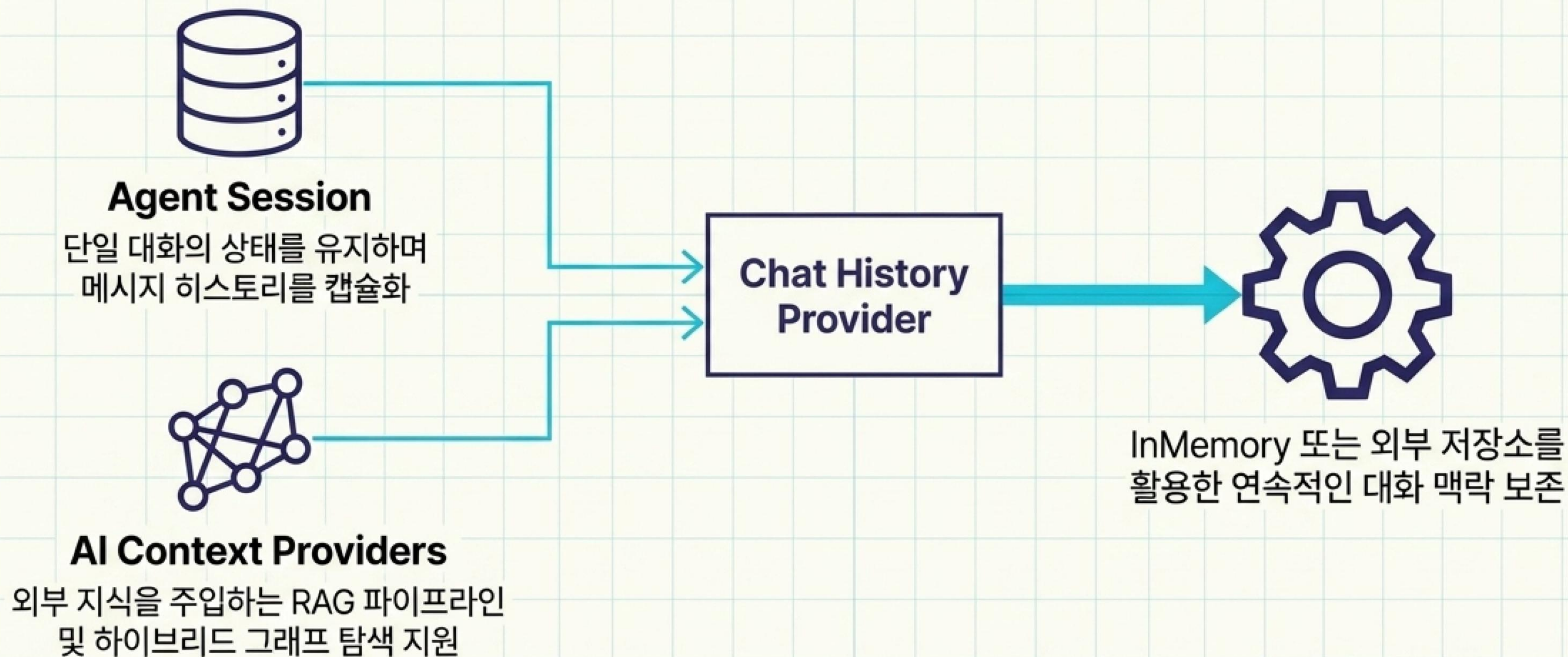


Model Context Protocol (MCP)

- 외부 도구 및 서비스와 에이전트를 연결하는 개방형 표준 프로토콜
- Local MCP: stdio를 통한 로컬 시스템 파일 및 셸 명령어 접근
- Hosted MCP: HTTP를 통한 원격 서비스 연동

도구 호출 기능은 에이전트의 단순 텍스트 생성을 넘어서 실제 행동 수행의 핵심

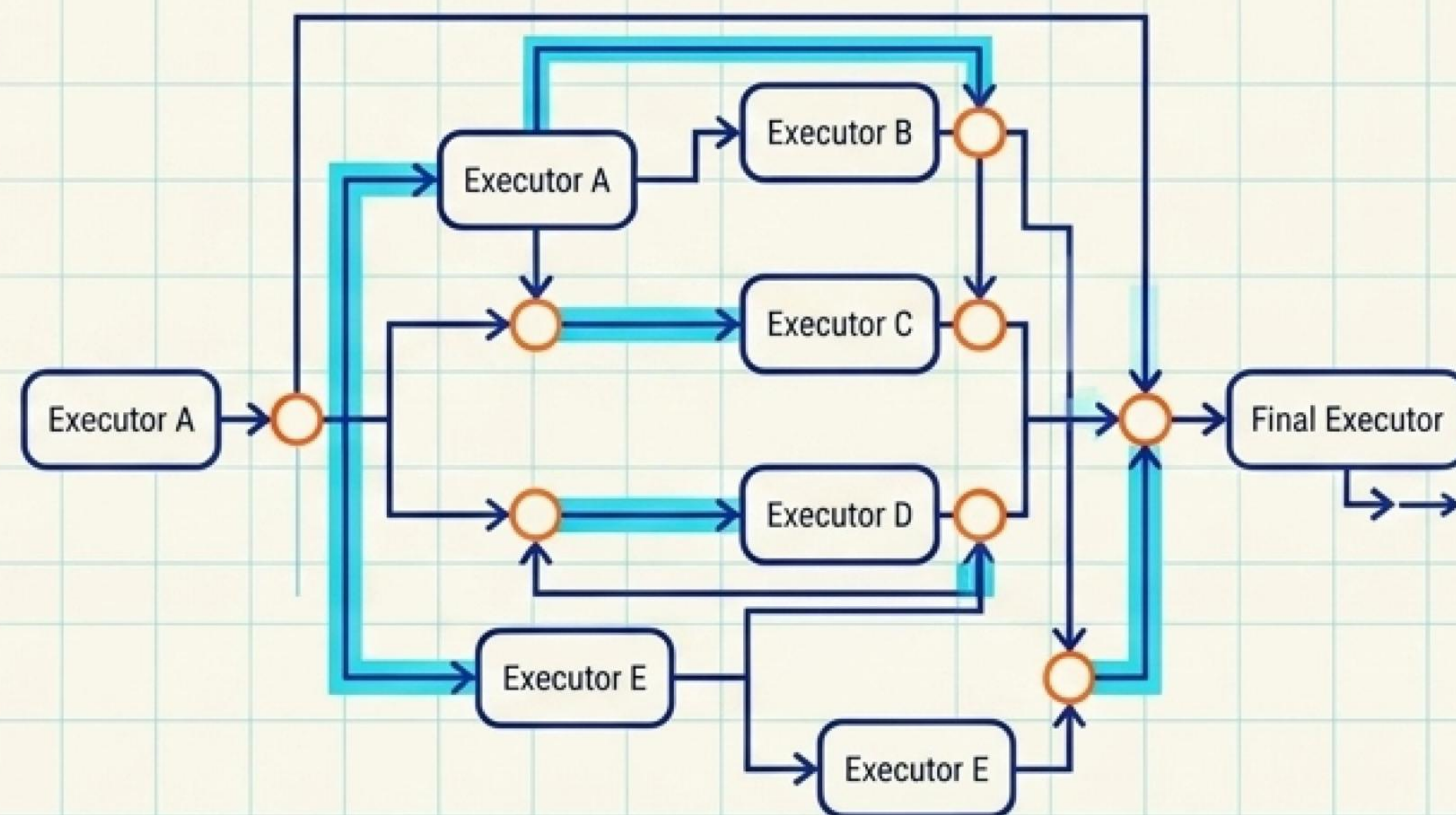
기억과 맥락의 주입: Session & Context



세션 직렬화(Serialization)를 통해 일시 중지된 장기 워크플로우의 완벽한 복원 가능

워크플로우: 그래프 기반 오케스트레이션

- 방향성 비순환 그래프(DAG) 형태의 구조적 멀티 에이전트 연결
- 타입 안전성(Type Safety): 런타임 오류를 방지하는 엄격한 메시지 라우팅 검증
- 체크포인트(Checkpointing): 긴 실행 시간의 프로세스를 위한 상태 저장 및 복구
- 선언적 YAML 기반 구성과 코드를 통한 동적 생성 모두 지원



개별 에이전트의 불확실성을 통제 가능한 기업용 비즈니스 프로세스로 편입

워크플로우 패턴 1: Sequential



에이전트들이 파이프라인 형태로 배치되어
순차적으로 태스크 실행

이전 에이전트의 출력결과가 다음 에이전트의
입력으로 전달

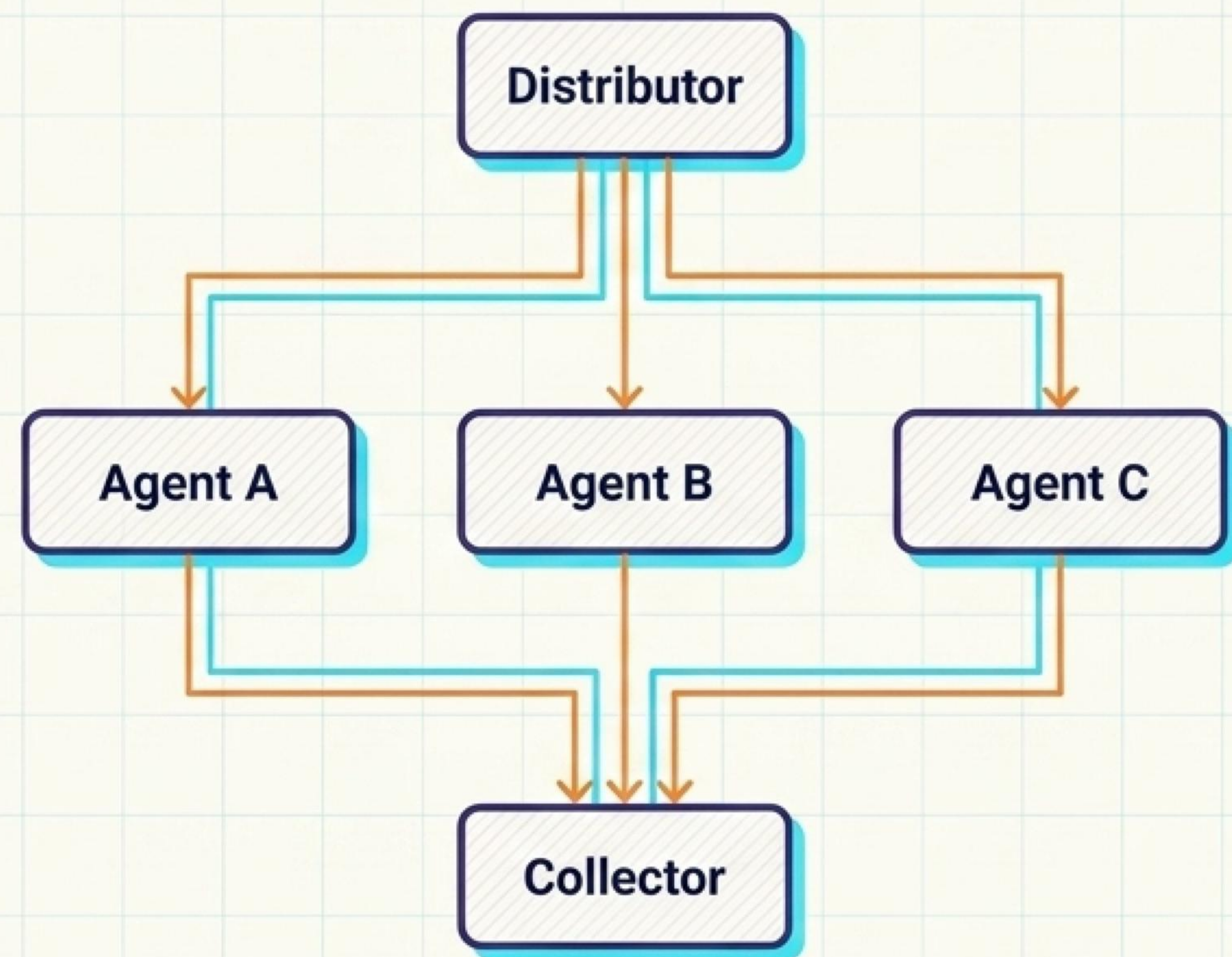
모든 이전 대화 기록이 파이프라인을 따라 흐르며
전체 맥락 유지

주요 활용 사례: 문서 검토 체인, 단계별 데이터
파이프라인, 다단계 추론 프로세스

AgentWorkflowBuilder.BuildSequential()을 통한 직관적인 구축

워크플로우 패턴 2: Concurrent

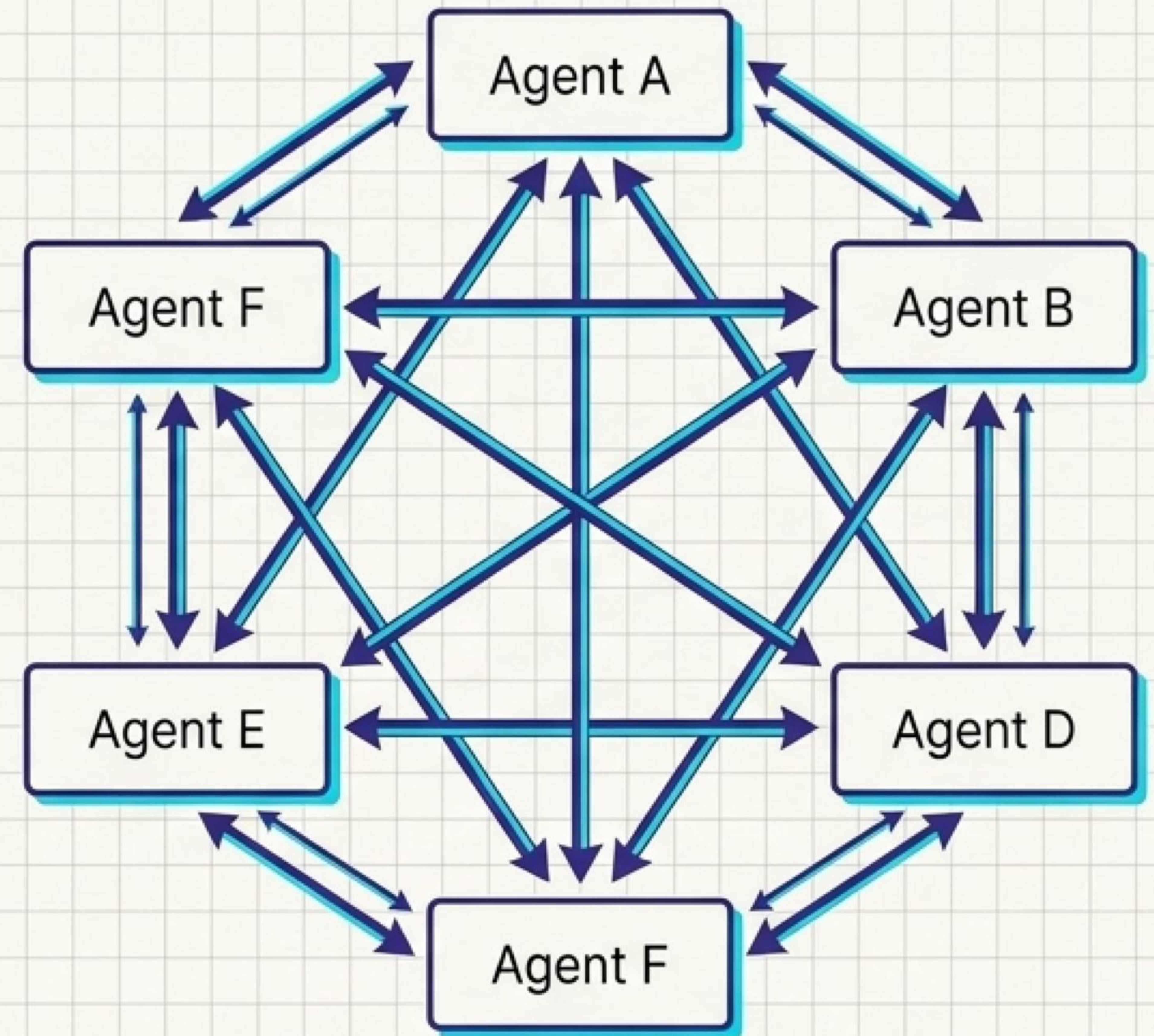
- 동일한 입력에 대해 여러 에이전트가 병렬로 독립적인 작업 수행
- 다양한 전문성을 가진 에이전트들의 결과를 최종적으로 수집 및 병합
- 실행 시간 단축 및 다각적 분석 결과 도출에 최적화
- 주요 활용 사례: 빠른 데이터 분석, 앙상블 추론, 브레인스토밍, 투표 시스템



AgentWorkflowBuilder.BuildConcurrent()로 자동 결과 집계 메커니즘 제공

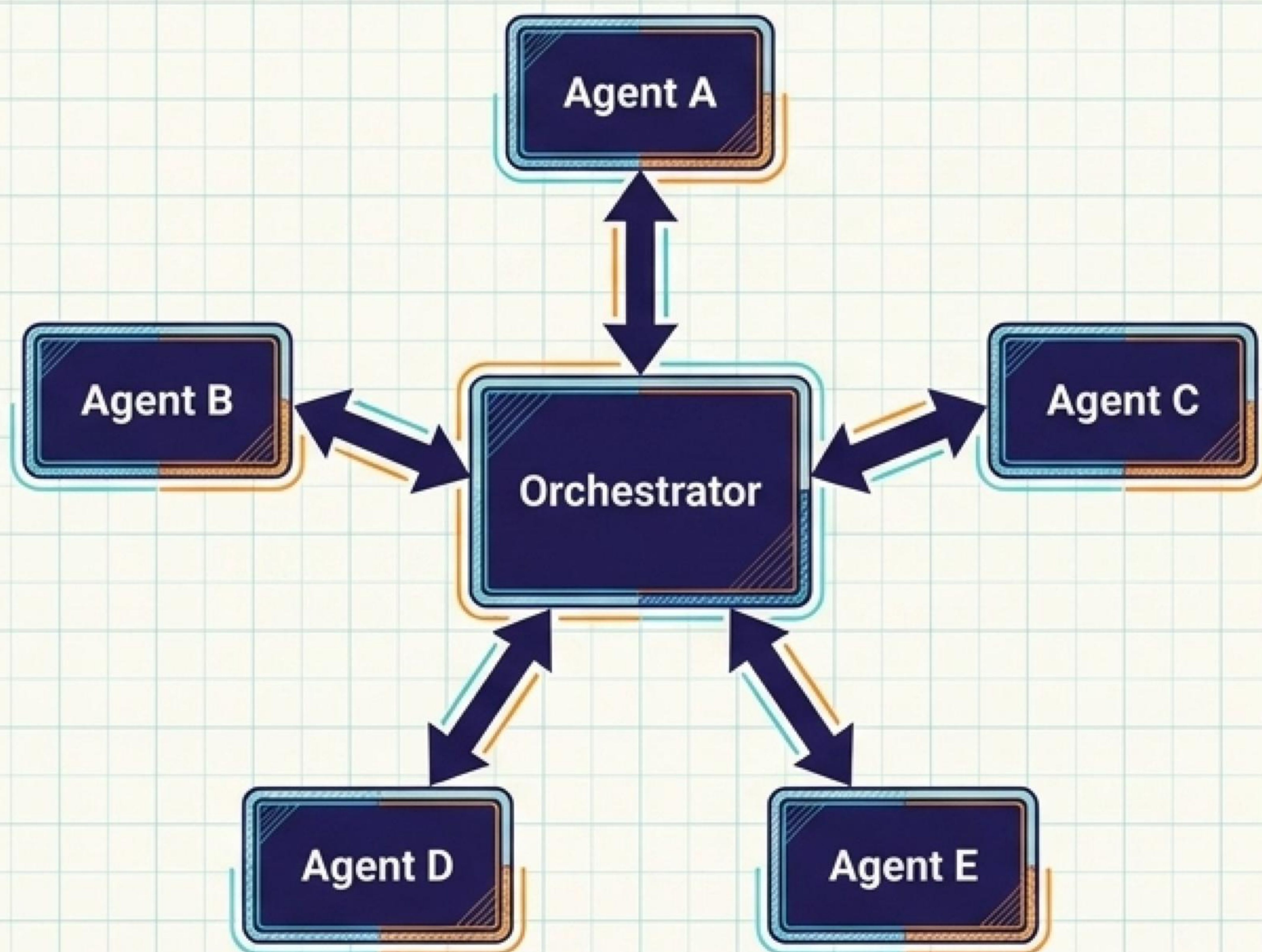
워크플로우 패턴 3: Handoff

- 대화의 맥락이나 사용자의 요청 내용에 따라 적합한 에이전트로 제어권 이관
- 동적인 라우팅을 통해 가장 전문성 있는 에이전트가 태스크 전담
- 세션은 분리되나 메시지 브로드캐스팅을 통해 전체 맥락의 일관성 유지
- 주요 활용 사례: 고객 지원 라우팅, 복합 전문가 시스템, 동적 태스크 위임



조건부 엣지(Edge)를 활용하여 정교한 제어권 전환 로직 설계

워크플로우 패턴 4: Group Chat



중앙 오케스트레이터가 대화의 흐름과 다음 발화자를 결정하는 구조

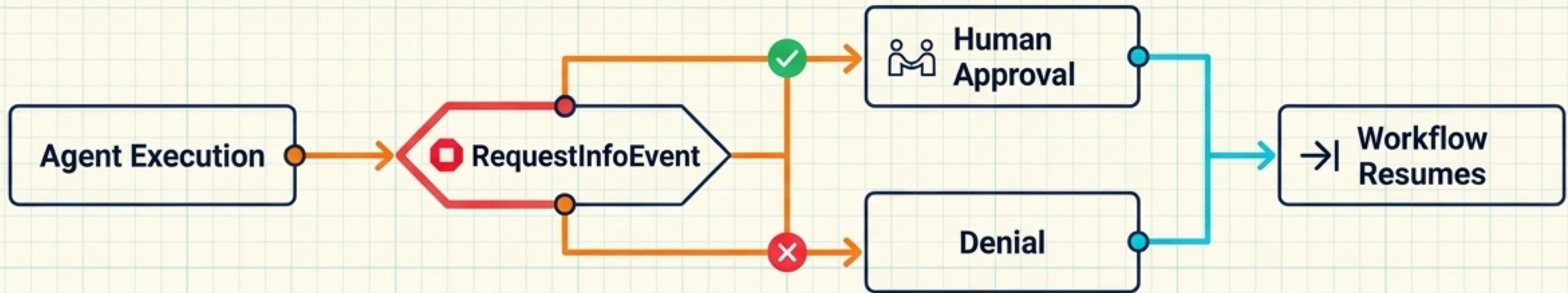
라운드 로빈, 프롬프트 기반 선택 등 다양한 발화자 선정 전략 지원

각 에이전트의 응답은 브로드캐스트되어 모두가 동일한 진행 상황 공유

주요 활용 사례: 복잡한 의사결정 토론, 반복적 산출물 개선, 다각도 분석

단일 방향 파이프라인의 한계를 극복하는 상호작용적 협업 패턴

Human-in-the-Loop: 신뢰와 통제



RequestPort 및 RequestInfoEvent

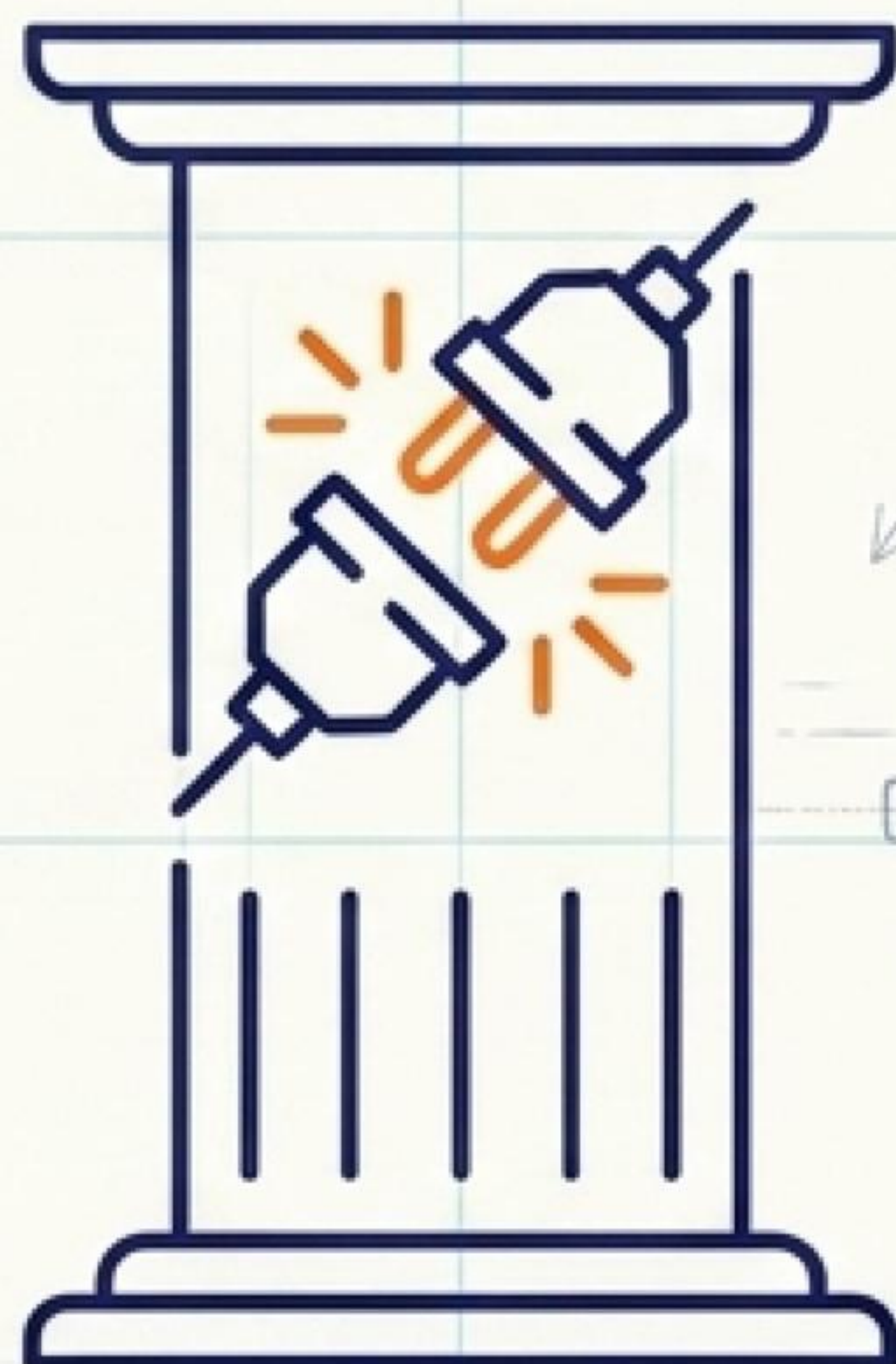
에이전트가 외부(사람)의 개입이나 승인이 필요할 때 워크플로우 일시 정지

ApprovalRequiredAIFunction

민감한 도구(DB 삭제, 결제 등) 호출 전
관리자의 명시적 승인 강제
서버 리소스 점유 없이 상태를 체크포인트로
저장하고 대기 모드 진입

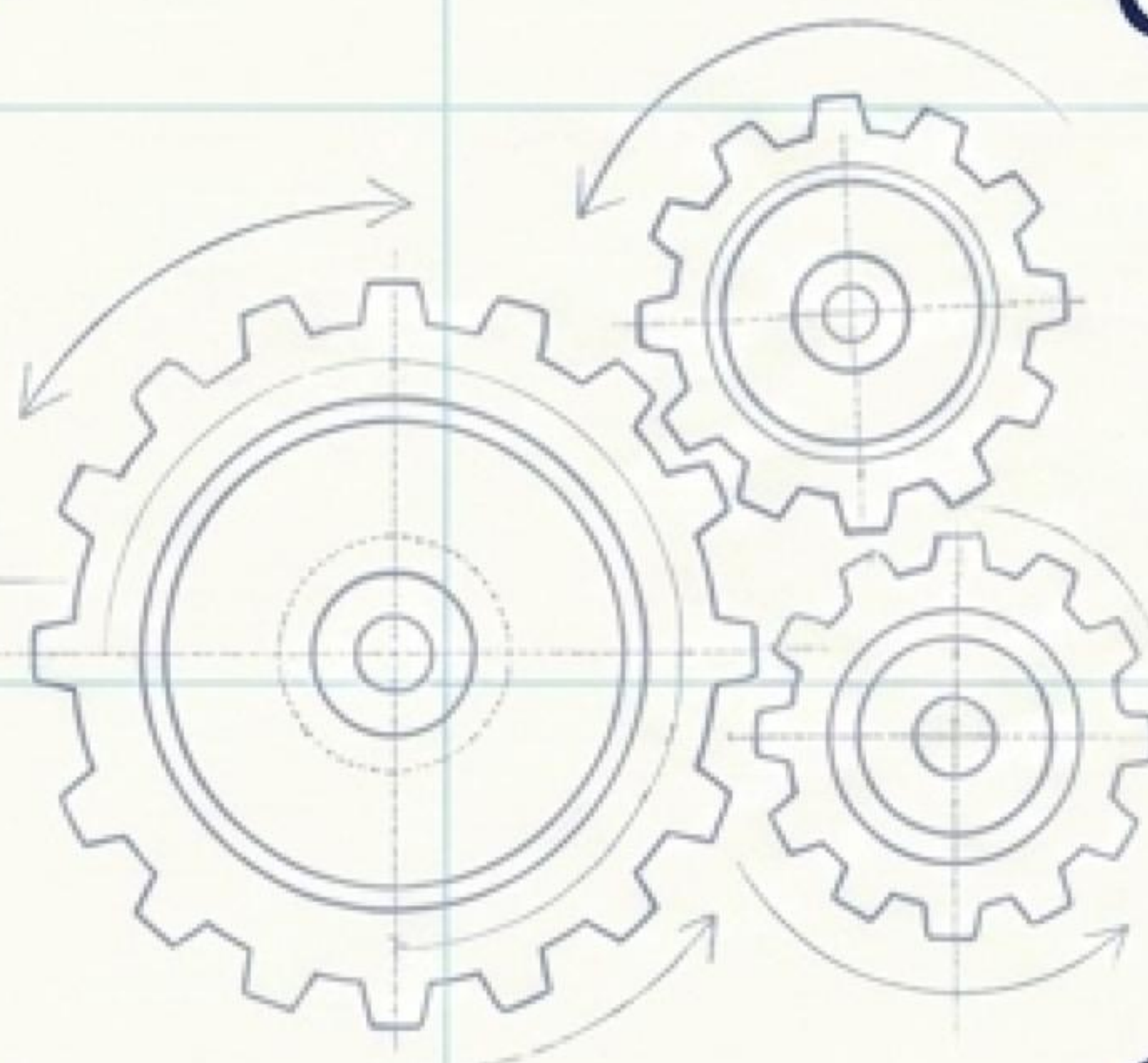
자율성의 확장과 비례하여 더욱 강력해진 안전장치 및 통제권

왜 Agent Framework와 Copilot인가



일관된 AI Agent 인터페이스

프로바이더를 변경해도
비즈니스 로직 수정 불필요



강력한 멀티 에이전트 조합

단순한 코드 생성을 넘어 Copilot을
워크플로우의 전문 노드로 편입



엔터프라이즈 에코시스템 통합

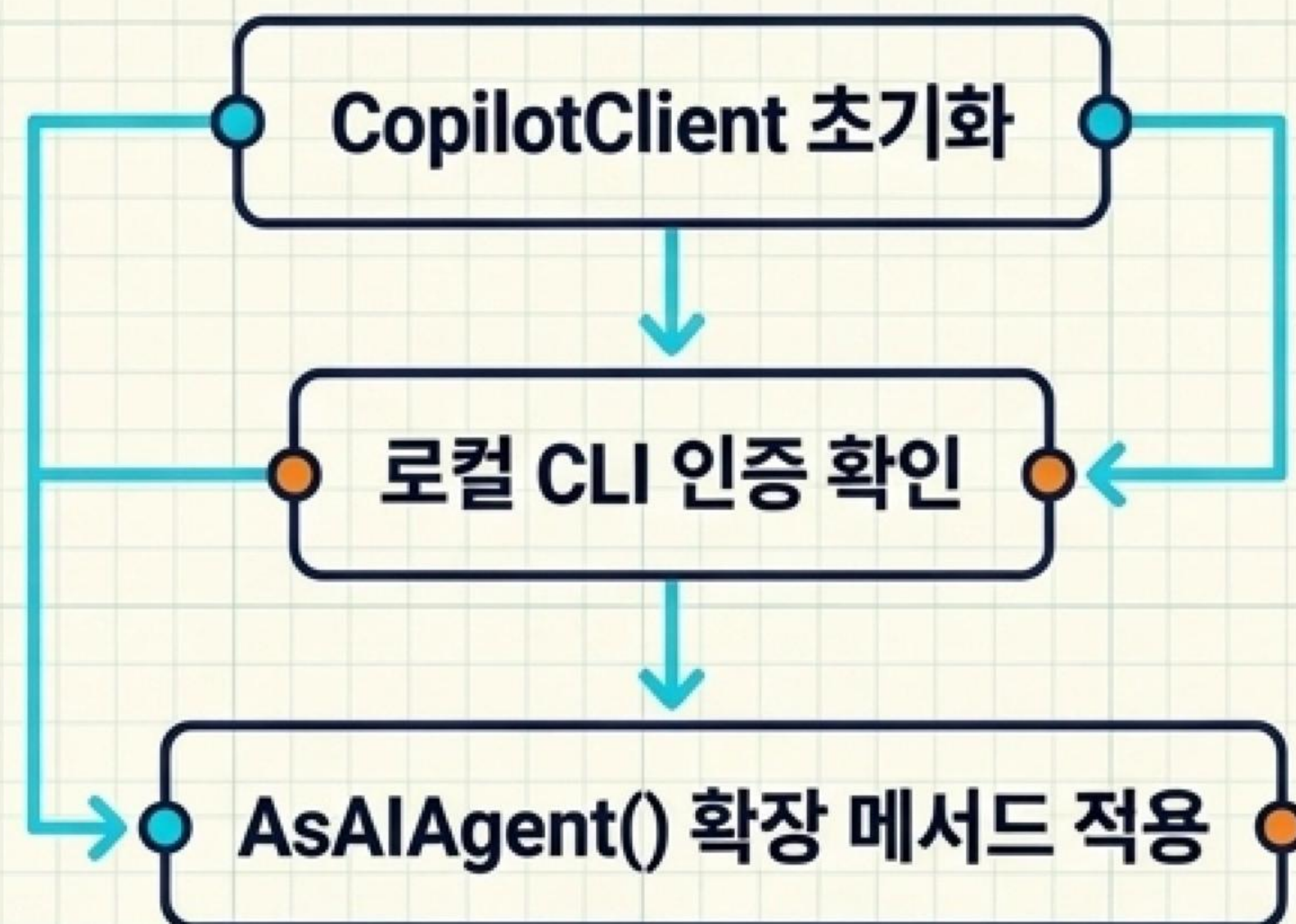
Function Tools, Streaming,
Middleware 등 프레임워크의
모든 고급 기능 활용

GitHub Copilot SDK를 프레임워크 위에 올려 엔터프라이즈 수준으로 확장

GitHub Copilot 에이전트 구축

```
import [redacted] from [redacted]

function [redacted]() {
  [redacted] = await [redacted]: [redacted]
  [redacted], [redacted]
  await [redacted] {
    return [redacted] ( [redacted] )
  }
  return [redacted]
}
```



단일 패키지 설치:
agent-framework-
github-copilot 패키지를
통한 즉각 연동

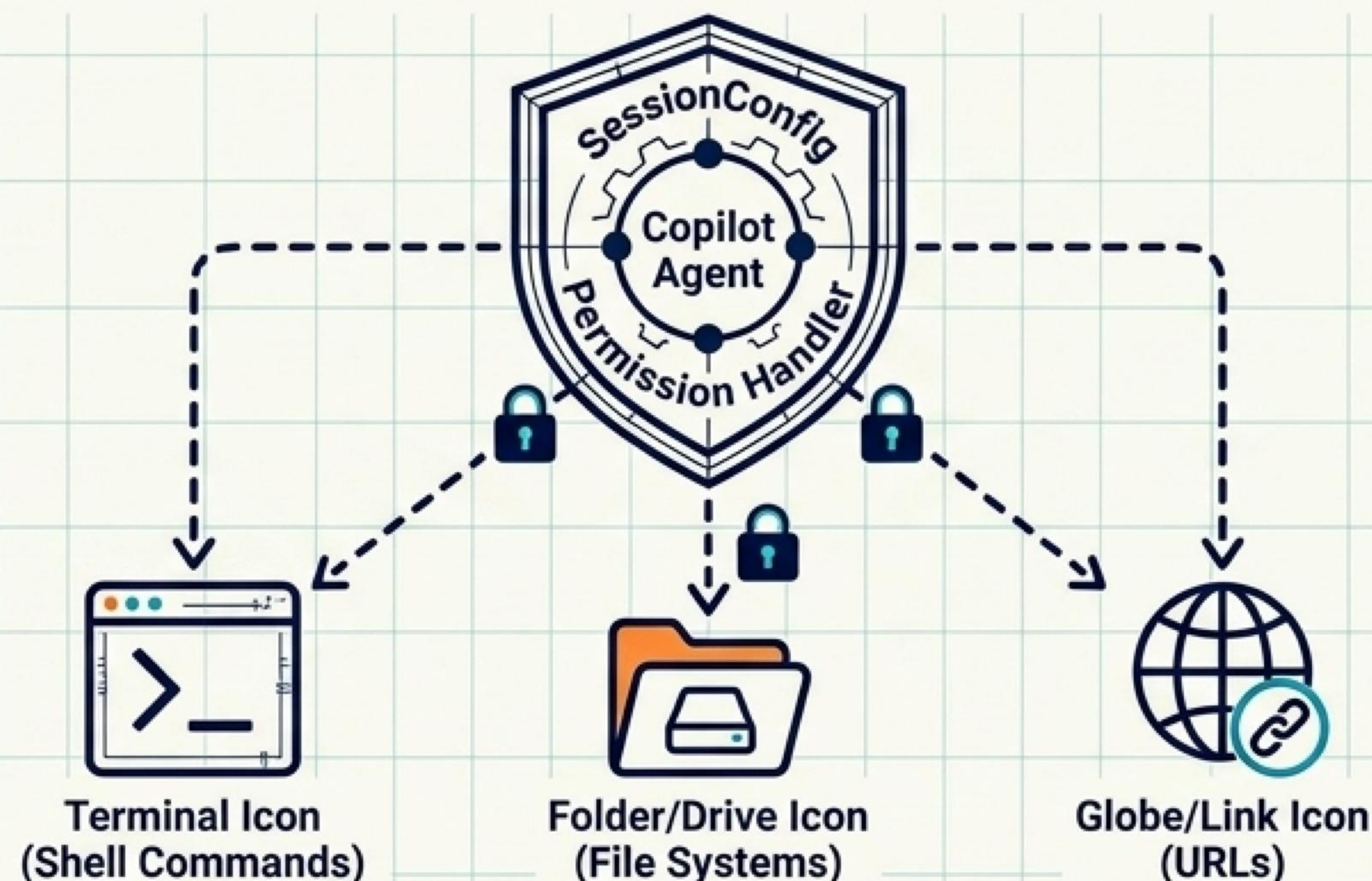
CopilotClient 초기화:
별도의 API 키 없이
로컬에 설치된
Copilot CLI 인증 활용

AsIAgent() 확장:
클라이언트를 표준
IAgent 인스턴스로
즉시 변환

**지시어(Instructions)와
커스텀 도구(Tools)를
주입하여 특화된 코드
전문가 생성**

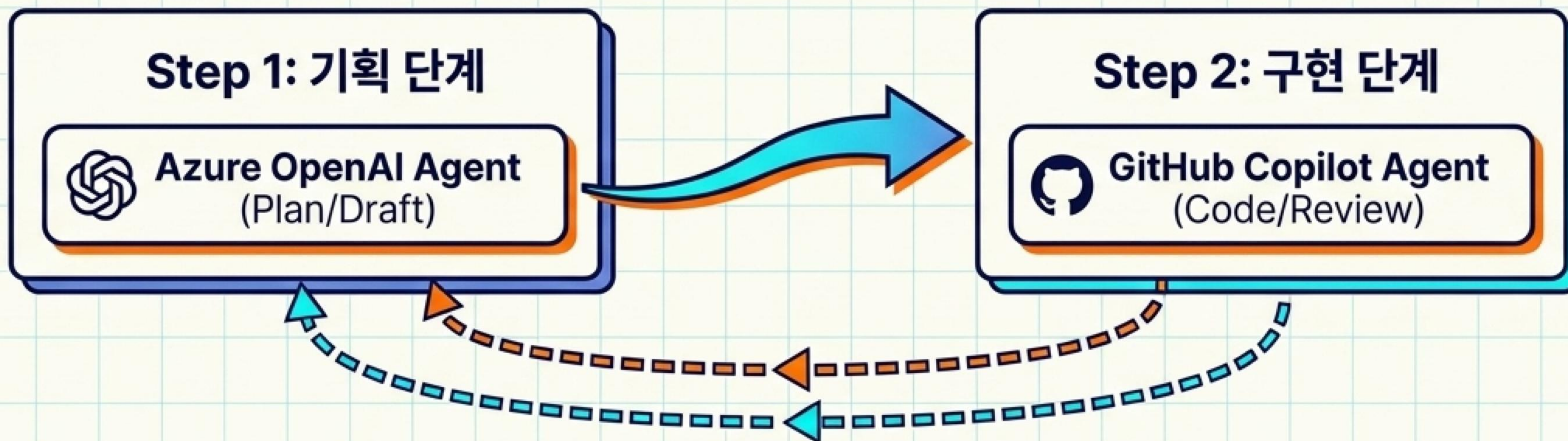
복잡한 인프라 설정 없이 개발자의 로컬 환경에서 즉시 구동

Copilot의 로컬 자원 접근



- 기본적으로 셸 커맨드, 파일 시스템, URL 접근은 보안을 위해 차단
- SessionConfig 기반 통제: OnPermissionRequest 핸들러를 통한 명시적 권한 부여
- 코드 분석 뿐만 아니라 실제 터미널 명령어 실행 및 로컬 파일 읽기/쓰기 수행
- 컨테이너화된 환경(Docker/Dev Container)과 결합하여 안전한 샌드박스 실행 권장

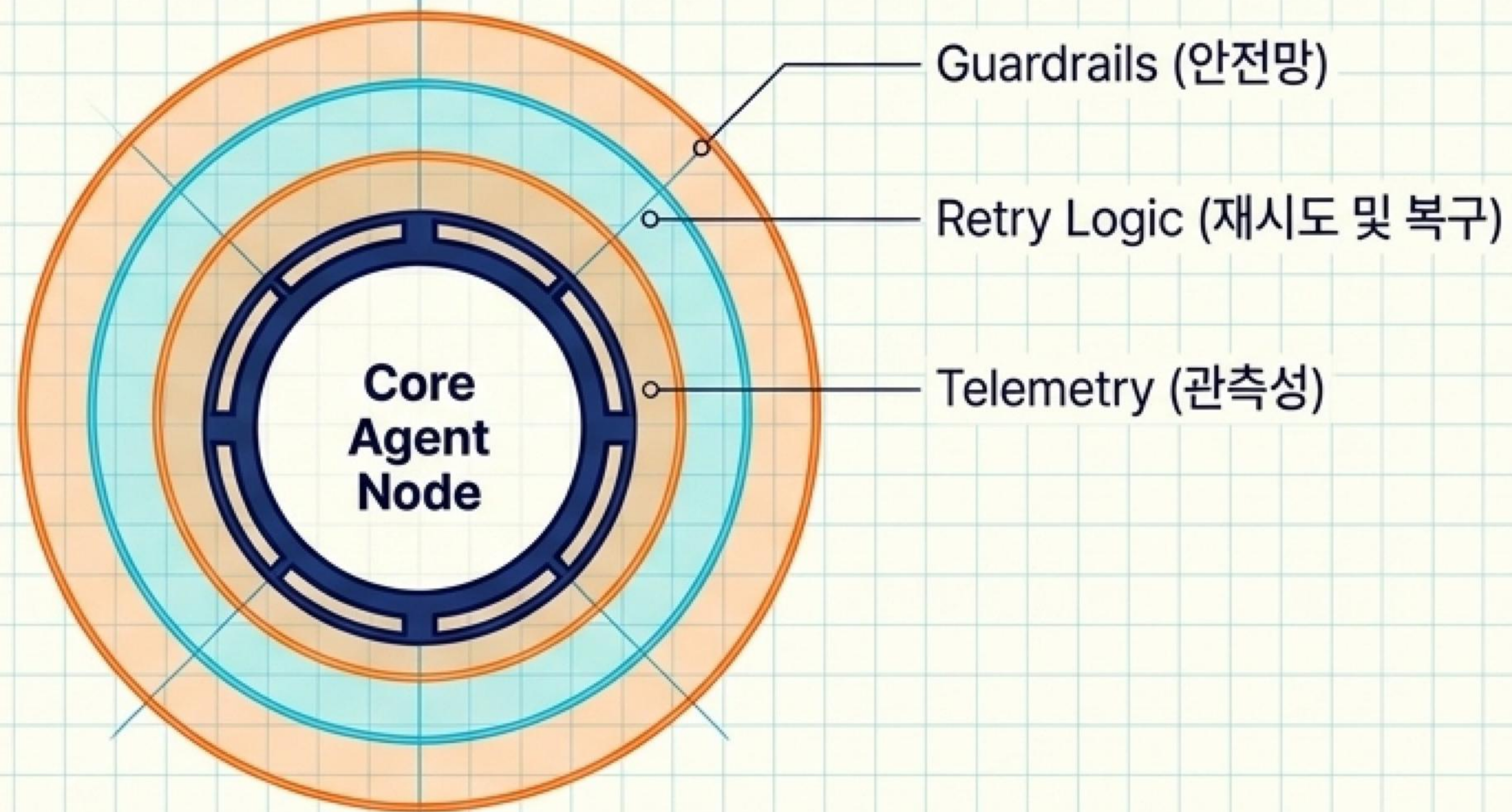
멀티 에이전트 워크플로우 속의 Copilot



- 서로 다른 프로바이더의 에이전트를 단일 워크플로우로 매끄럽게 연결
- Azure OpenAI 에이전트가 요구사항을 분석하고 아키텍처 초안 작성
- **GitHub** Copilot 에이전트가 초안을 바탕으로 실제 코드 작성 및 리뷰 수행
- Provider가 달라도 run() 인터페이스와 메시지 규격이 완벽히 호환

단일 모델의 한계를 넘어선 최적의 프로바이더 앙상블 구성

횡단 관심사 제어: Middleware



Guardrails: 비즈니스 규칙 적용 및 유해 콘텐츠 필터링, 정책 위반 시 조기 차단

Retry Logic: 일시적 네트워크 오류 처리 및 Graceful Degradation 자동 적용

Telemetry: 토큰 사용량, 지연 시간 추적 및 민감 데이터 마스킹 로깅

코어 비즈니스 로직을 오염시키지 않고 에이전트의 신뢰성과 보안 확보

표준 통신 프로토콜: A2A



- 서로 다른 서비스, 언어, 조직 경계를 넘나드는 에이전트 간 표준 HTTP 통신 규격
- Agent Card를 통한 에이전트 기능 광고 및 디스커버리
- 장시간 작업 처리를 위한 비동기 및 스트리밍 응답 지원

개별 환경에 갇힌 스크립트를 넘어 엔터프라이즈 규모의 상호 연결망 구축

데모 예제

<https://github.com/hexideacube/github-copilot-maf>



LIVE DEMO

코드 분석 → 수정 → 테스트 에이전트 직접 구축

`pip install agent-framework-github-copilot` · Python 3.10+

데모

1	기본 에이전트 생성	CopilotClient → AsAIAgent() → RunAsync()
2	코드 분석 도구 추가	AIFunctionFactory.Create() → tools 파라미터
3	스트리밍 출력	RunStreamingAsync() → 실시간 토큰 출력
4	멀티턴 세션	CreateSessionAsync() → 대화 맥락 유지
5	MCP 파일 작업	Shell & FileSystem MCP 서버 연결

리소스 & 다음 단계

공식 문서	learn.microsoft.com/agent-framework
GitHub Copilot SDK 통합 블로그	devblogs.microsoft.com/agent-framework
GitHub Copilot CLI 설치	github.com/github/copilot-sdk
마이그레이션 가이드 (SK·AutoGen)	learn.microsoft.com/agent-framework/migration
NuGet 패키지	nuget.org — Microsoft.Agents.AI.GitHub.Copilot

감사합니다.

질문은 언제든지 환영합니다

이정구 dennis.lee@ideacube.co.kr